Intermediate (measure the sequential version) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Two functions, chained directly — `write(research(topic))` — is the entire structure. There's no graph here yet; that's deliberate, since a fixed two-step pipeline doesn't need LangGraph to prove the pattern works. (You'll build the graph version of a similar pipeline as part of the supervisor comparison and Project 4.)

What you're adding on top of the plain functions is measurement:

- **Wall-clock time** — wrap each call in `time.perf_counter()` before/after, and again around the whole thing. `time.perf_counter()` is meant specifically for measuring elapsed durations and isn't affected by system clock adjustments, unlike `time.time()`.
- **Token usage** — most LLM responses carry usage metadata (`response.usage_metadata`, a dict-like with `input_tokens`, `output_tokens`, `total_tokens`) — read it instead of estimating with a separate tokenizer; the provider already tells you exactly what it billed you for.
- **Keeping research's numbers separate from write's numbers** — you need both stages' individual counts, not just a combined total, or you can't tell which stage is actually the expensive one.

Sketch the two functions and where the timing/token-counting code goes around each one, then go one step further: think about what happens the moment you want to compare this pipeline's numbers against something else — the supervisor version in `supervisor_compare`, or a different topic run tomorrow. A loose tuple of numbers, or three separately-returned values, gets confusing fast once you're comparing two runs side by side, and it's easy to accidentally compare research's tokens against write's seconds by mistake.

The real design question isn't just "measure each stage" — it's "what shape should these numbers live in, so a later comparison can't silently mix them up, and so the whole pipeline's report is

reusable instead of one-off print statements?"

The extra pieces needed:

- A small typed result per stage (a `@dataclass` with `text`, `tokens`, `seconds` fields), instead of a raw tuple — so a caller reading `result.tokens` can't confuse it with `result.seconds`.
- A separate `run_sequential(topic)` function that owns the "run the whole pipeline and report on it" concern, distinct from what each stage itself does — so either the pipeline or an individual stage can be tested or reused alone.
- Thinking one step ahead to `supervisor_compare`: whatever shape you settle on here is the shape you'll want the supervisor version's report to match, field for field, so the two are directly comparable without translation.

Sketch this hardened version yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
def research(topic: str) -> dict:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

def write(research_text: str) -> dict:
    start = time.perf_counter()
    prompt = f"Write a 3-sentence summary of this:\n\n{research_text}"
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }
```

```
# architecture_comparison_practice.py
def main():
    topic = "climate change"
    total_start = time.perf_counter()
```

```
    research_result = research(topic)
    write_result = write(research_result["text"])

    total_seconds = time.perf_counter() - total_start
    total_tokens = research_result["tokens"] + write_result["tokens"]

    r_tokens, r_seconds = research_result["tokens"], research_result["seconds"]
    w_tokens, w_seconds = write_result["tokens"], write_result["seconds"]

    print(f"research: {r_tokens} tokens, {r_seconds:.2f}s")
    print(f"write: {w_tokens} tokens, {w_seconds:.2f}s")
    print(f"total: {total_tokens} tokens, {total_seconds:.2f}s")
```

Run it, then compare these real numbers against your Basic-exercise guess, then go one step further, into the typed, reusable shape below:

```
make a StageResult dataclass with fields: text, tokens, seconds

function research(topic) -> StageResult:
    time it, call the model, wrap the result in StageResult

function write(research_text) -> StageResult:
    time it, call the model, wrap the result in StageResult

function run_sequential(topic) -> dict:
    time the whole thing
    research_result = research(topic)
    write_result = write(research_result.text)
    return a report dict: research, write, total_tokens, total_seconds
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# architecture_comparison_practice.py
import time
from dataclasses import dataclass

@dataclass
class StageResult:
    text: str
    tokens: int
    seconds: float

def research(topic: str) -> StageResult:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
```

```
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    call_tokens = response.usage_metadata["total_tokens"]
    return StageResult(response.content, call_tokens, elapsed)

def write(research_text: str) -> StageResult:
    # your turn: same shape as research() above, but for the write stage
    ...

def run_sequential(topic: str) -> dict:
    total_start = time.perf_counter()
    research_result = research(topic)
    write_result = write(research_result.text)
    total_seconds = time.perf_counter() - total_start

    return {
        "research": research_result,
        "write": write_result,
        "total_tokens": research_result.tokens + write_result.tokens,
        "total_seconds": total_seconds,
    }
```

Fill in `write()` yourself, then compare your written reflection ("was I right about which stage costs more?") against the Solution.

Full solution: Show me the solution